Relational Database vs NoSQL Database

A comparison of data models and use cases using the hotel management system

Prepared by: Hemanth Krishna S

Project: Hotel Management System. Output convention: result tables are expected outputs from the fixed rows in the shared SQL script; they are not screenshots of MySQL execution.

1. Purpose

A relational database stores related facts in tables connected through keys. NoSQL is a broad family of non-relational models, including document, key-value, wide-column, and graph databases. The appropriate choice depends on data shape, access patterns, consistency requirements, and scale.

2. Comparison for the hotel project

Dimension	Relational design	Document / NoSQL design
Data model	Separate tables for guests, rooms, bookings, services, and payments; keys connect records.	A guest or booking document could embed selected details or service items.
Consistency	Foreign keys, unique keys, checks, and transactions protect related booking and payment changes.	Consistency depends on the product and transaction boundaries; duplicated fields may need coordinated updates.
Scaling	Can begin on one server and scale using replicas, partitioning, or sharding when needed.	Many products support horizontal partitioning; access patterns still guide the data model.
Query style	SQL joins and aggregates support occupancy, revenue, guest history, and service reports.	Document reads can efficiently return a booking and its embedded details; cross-document reports may need aggregation.
Schema change	Declared columns and constraints support predictable validation; migrations manage changes.	Documents can vary by record, useful for optional preferences or changing metadata.
Good fit	Bookings, room allocation, payments, and accounting where relationships and integrity matter.	Flexible guest preferences, event logs, or a read-heavy search/catalog view.

3. Hotel example

The relational model stores a guest once, each physical room once, and each booking as a separate transaction. A booking references a guest and room, while payments reference the booking. This supports reports such as active bookings by hotel and revenue by payment status.

Illustrative document shape

```
{
  "bookingId": 1001,
  "guest": {"guestNo": "GST001", "name": "Ananya Rao"},
  "hotel": "Lakeview Residency",
  "room": {"number": "101", "type": "Standard"},
  "checkIn": "2026-10-20",
  "checkOut": "2026-10-24",
  "services": [
    {"name": "Breakfast", "quantity": 2},
    {"name": "Laundry", "quantity": 1}
  ]
}
```

This JSON is an illustration only. The submitted implementation remains the normalized MySQL schema used in Assignments 5–9.

4. Choosing a technology in practice

Question	Why it matters here
Are relationships essential?	A booking must refer to an existing guest and room; payments must refer to an existing booking.
Do reports cross entities?	Occupancy, guest history, and revenue reports require joins and aggregation.
How variable is the data?	Optional preferences may be flexible, while booking and payment fields require validation.
What is the expected scale?	A student hotel project can begin with one relational database and revisit scaling if usage grows.

5. Decision

MySQL 8.0 is selected for this BCA project because booking, room, guest, and payment relationships are central and the assignments require DDL, constraints, joins, and aggregate reports. A document database could complement a larger search or personalization service, but it would not remove the need to define consistency rules.

6. References

MySQL 8.0 Reference Manual, InnoDB and the ACID Model: https://dev.mysql.com/doc/refman/8.0/en/mysql-acid.html

MongoDB Manual, Embedded Data: https://www.mongodb.com/docs/manual/data-modeling/embedding/

7. Screenshot guidance

This comparison is conceptual, so a query screenshot is not required. If execution evidence is requested, run SELECT VERSION(); in MySQL Workbench and capture the result.